SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1704**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I Bhavya S(192524376)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Bhavya S

Scenario 1: AI-Powered Drone for Emergency Medicine Delivery

Scenario Understanding

A government uses AI-powered drones to deliver emergency medicines in a flood-affected region. The environment changes frequently due to blocked roads, floods, and bad weather. The drone receives partial real-time information about blocked paths and must quickly find the safest and fastest route. The objective is to minimize delivery delay while ensuring safety.

i. Greedy Best-First Search (GBFS): Behavior, Strengths and Weaknesses

How GBFS Works

Greedy Best-First Search selects the path that appears closest to the destination based only on the heuristic value **h(n)** (straight-line distance). It ignores the cost already traveled.

Evaluation Function:

$$f(n) = h(n)$$

Behavior in this Scenario

- The drone always chooses the route that seems nearest to the hospital.
- It reacts quickly because it only considers heuristic distance.
- If a road becomes blocked unexpectedly, it searches for another path toward the goal.

Strengths

- Fast decision-making.
- Low computational cost.
- Suitable when emergency response is more important than finding the perfect route.

Weaknesses

- May choose unsafe or expensive routes.
- Ignores weather and terrain costs.
- Does not always produce the shortest or safest path.
- Can waste time if it repeatedly encounters blocked paths.

Example

Route	Heuristic Distance	Actual Cost
A	5 km	High (Flooded)

Route Heuristic Distance Actual Cost

B 7 km Low (Safe)

GBFS selects **Route A** because it has the smallest heuristic, even though it is riskier

ii. A Search: Balancing Cost and Heuristic*

How A Works*

A* considers both:

- Actual cost traveled **g(n)**
- Estimated remaining distance **h(n)**

Evaluation Function:

$$f(n) = g(n) + h(n)$$

Behavior in this Scenario

The drone evaluates:

- Distance already travelled
- Weather conditions
- Terrain difficulty
- Remaining distance

Instead of selecting only the nearest path, it selects the route with the lowest total estimated cost.

Advantages

- Finds the optimal route when the heuristic is admissible.
- Avoids expensive flooded or risky areas.
- Produces safer deliveries.
- Balances speed and travel cost.

Disadvantages

- Requires more computation.
- Uses more memory.
- Slightly slower than GBFS.

Example

Route	g(n)	h(n)	f(n)=g+h
A	10	5	15
B	6	8	14

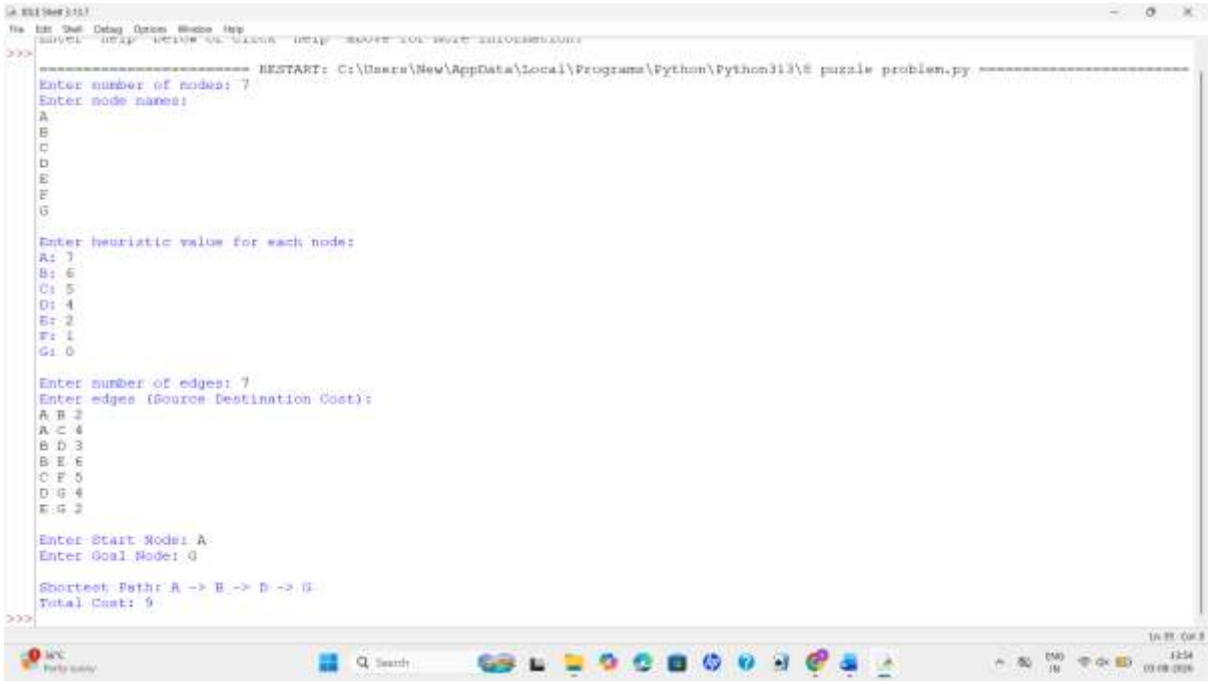
A* chooses **Route B** because its total cost is lower.

Python Code:

```
import heapq

# Manual input
n = int(input("Enter number of nodes: "))

graph = {}

print("Enter node names:")
nodes = []
for i in range(n):
    node = input()
    nodes.append(node)
    graph[node] = []

print("\nEnter heuristic value for each node:")
heuristic = {}
for node in nodes:
    heuristic[node] = int(input(f"{node}: "))

e = int(input("\nEnter number of edges: "))

print("Enter edges (Source Destination Cost):")
for i in range(e):
    u, v, cost = input().split()
    graph[u].append((v, int(cost)))
    graph[v].append((u, int(cost)))   # Remove this line if graph is directed
```

```
start = input("\nEnter Start Node: ")
goal = input("Enter Goal Node: ")

def astar(start, goal):
    pq = []
    heapq.heappush(pq, (heuristic[start], 0, start, [start]))
    visited = set()

    while pq:
        f, g, current, path = heapq.heappop(pq)

        if current == goal:
            print("\nShortest Path:", " -> ".join(path))
            print("Total Cost:", g)
            return

        if current in visited:
            continue

        visited.add(current)

        for neighbor, cost in graph[current]:
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbor]
                heapq.heappush(pq, (new_f, new_g, neighbor, path + [neighbor]))

    print("No Path Found")

astar(start, goal)
```

Output



iii. Impact of Heuristic Accuracy

The heuristic estimates the remaining distance to the goal.

For Greedy Best-First Search

- Highly dependent on heuristic accuracy.
- Accurate heuristic → Faster navigation.
- Inaccurate heuristic → Wrong path selection and unnecessary delays.

For A*

- Uses both heuristic and actual path cost.
- Less affected by heuristic errors.
- Still performs well even if the heuristic is not perfect.
- If the heuristic is admissible (never overestimates), A* guarantees the optimal solution.

Comparison

Heuristic Accuracy	**GBFS**	**A***
Accurate	Very Fast	Fast & Optimal
Moderate	May make mistakes	Usually optimal

Heuristic Accuracy	GBFS	A*
Poor	Performs poorly	Better than GBFS

iv. Effect of Dynamic Changes (Blocked Paths)

Flood conditions may suddenly block roads or create new obstacles.

Greedy Best-First Search

- May repeatedly choose blocked paths.
- Needs frequent replanning.
- Performance decreases in highly dynamic environments.

A*

- Updates path cost when new information is received.
- Searches for an alternative route considering total cost.
- More reliable in changing environments.
- Handles unexpected obstacles more effectively.

Impact

Dynamic Change	GBFS	A*
Blocked Road	May fail repeatedly	Finds better alternative
Heavy Rain	Ignores extra cost	Considers increased cost
New Obstacle	Frequent rerouting	Efficient replanning

v. Recommended Algorithm

Recommended: A Search Algorithm*

Justification

Real-Time Performance

- Slightly slower than GBFS but still fast enough for emergency routing.

Optimality

- Finds the minimum-cost path by considering both travel cost and heuristic estimate.

Safety

- Avoids dangerous flooded or high-risk routes.

- Considers weather and terrain conditions.

Adaptability

- Adjusts efficiently when blocked paths or changing conditions are detected.

Comparison

Criteria	Greedy Best-First Search	A* Search
Speed	Very High	High
Optimal Solution	No	Yes
Safety	Low	High
Handles Dynamic Changes	Moderate	Excellent
Uses Travel Cost	No	Yes

Final Decision

A* is the most suitable algorithm for this emergency medicine delivery scenario because it balances speed, travel cost, and safety. Although Greedy Best-First Search is faster, it may choose risky or inefficient routes. A* provides reliable, cost-effective, and safer navigation, making it the best choice for disaster-response operations.

Conclusion

In a flood-affected environment with changing road conditions and weather, Greedy Best-First Search offers quick decisions but may produce unsafe or inefficient routes because it relies only on the heuristic estimate. A* combines the actual travel cost with the heuristic estimate to identify safer and more efficient paths. Considering real-time constraints, optimality, adaptability, and safety, A* is the recommended algorithm for AI-powered emergency medicine delivery drones.

Scenario 2: Smart City Traffic Signal Optimization

Scenario Understanding

A smart city uses an AI system to optimize traffic signal timings across hundreds of intersections. The objectives are to:

- Minimize total waiting time
- Reduce fuel consumption
- Decrease traffic congestion

The system continuously receives real-time traffic data. Since the number of possible signal timings is extremely large and traffic conditions change constantly, finding the best solution is a complex optimization problem.

Problem Formulation as an Optimization Problem

Objective Function

The AI system aims to minimize:

- Total vehicle waiting time
- Fuel consumption
- Traffic congestion

Decision Variables

- Green light duration
- Red light duration
- Signal sequence
- Coordination between nearby intersections

Constraints

- Minimum and maximum signal timings
- Pedestrian crossing time
- Emergency vehicle priority
- Traffic safety rules

Why Traditional Search is Inefficient

Traditional search algorithms such as BFS or DFS explore many possible signal

combinations. With hundreds of intersections, the search space becomes enormous, making them:

- Slow
- Memory-intensive
- Unsuitable for real-time traffic control

Optimization algorithms are preferred because they search for good solutions without exploring every possibility.

i. Hill Climbing Algorithm and Its Limitations

How Hill Climbing Works

Hill Climbing starts with an initial traffic signal plan and makes small changes. If a new timing reduces waiting time or congestion, it accepts the change; otherwise, it rejects it. This process continues until no further improvement is found.

Working in This Scenario

- Start with current signal timings.
- Slightly adjust green/red durations.
- Measure traffic performance.
- Keep changes that improve traffic flow.
- Stop when no better neighboring solution exists.

Advantages

- Simple to implement
- Fast execution
- Suitable for real-time local optimization
- Low memory usage

Limitations

- Can get stuck in local optimum
- Cannot always find the best global solution
- Performance depends on the starting solution

ii. Local Maxima, Plateaus, and Ridges

1. Local Maximum

A local maximum is a solution that is better than nearby solutions but not the best overall.

Traffic Example:

The AI improves one intersection, but congestion remains high in neighboring intersections.

2. Plateau

A plateau is a flat region where many neighboring solutions have the same quality.

Traffic Example:

Changing signal timings by a few seconds does not improve waiting time, making it difficult for the algorithm to decide the next move.

3. Ridge

A ridge is a narrow path where improvement requires multiple coordinated changes.

Traffic Example:

Reducing congestion requires adjusting signals at several intersections together. Changing only one signal does not improve traffic flow.

Summary

Problem	**Real-World Interpretation**
Local Maximum	Good but not best traffic plan
Plateau	No visible improvement after changes
Ridge	Requires multiple coordinated signal changes

iii. Simulated Annealing and Genetic Algorithm

Simulated Annealing

Working

Simulated Annealing sometimes accepts worse solutions temporarily. This helps the algorithm escape local maxima and continue searching for better solutions.

Advantages

- Escapes local optima
- Finds better global solutions
- Suitable for dynamic environments

Disadvantages

- Slower than Hill Climbing

- Requires careful temperature control

Genetic Algorithm (GA)

Working

Genetic Algorithm treats each traffic signal plan as a chromosome.

Steps:

1. Generate multiple signal timing plans.
2. Evaluate each plan using a fitness function.
3. Select the best plans.
4. Apply crossover and mutation to create new plans.
5. Repeat until an optimal solution is found.

Advantages

- Searches many solutions simultaneously
- Avoids local maxima
- Handles very large search spaces
- Produces high-quality solutions

Disadvantages

- Higher computational cost
- Requires parameter tuning

Comparison

Feature	**Hill Climbing**	**Simulated Annealing**	**Genetic Algorithm**
Speed	High	Medium	Medium
Escapes Local Maximum	No	Yes	Yes
Global Optimization	Poor	Good	Excellent
Large Search Space	Poor	Good	Excellent
Dynamic Traffic	Moderate	Good	Excellent

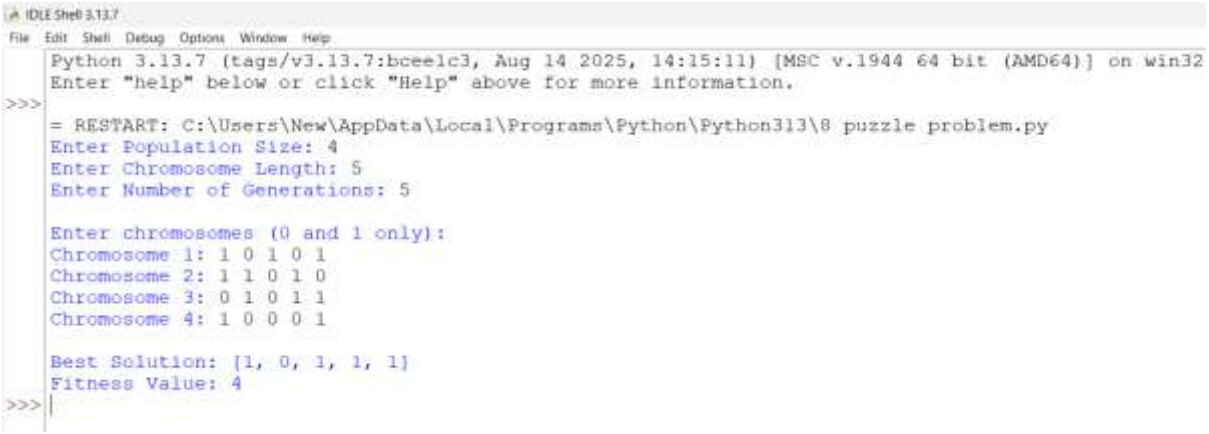
Python Code:

import random

```
# Fitness Function
def fitness(chromosome):
    return sum(chromosome)

# Crossover Function
def crossover(parent1, parent2):
    point = random.randint(1, len(parent1) - 1)
    child = parent1[:point] + parent2[point:]
    return child

# Mutation Function
def mutation(child):
    pos = random.randint(0, len(child) - 1)
    child[pos] = 1 - child[pos]
    return child

# Manual Input
population_size = int(input("Enter Population Size: "))
chromosome_length = int(input("Enter Chromosome Length: "))
generations = int(input("Enter Number of Generations: "))

population = []

print("\nEnter chromosomes (0 and 1 only):")
for i in range(population_size):
    chromosome = list(map(int, input(f"Chromosome {i+1}: ").split()))
    population.append(chromosome)

# Genetic Algorithm
for g in range(generations):

    population = sorted(population, key=fitness, reverse=True)
```

```
    parent1 = population[0]
    parent2 = population[1]

    child = crossover(parent1, parent2)
    child = mutation(child)

    population[-1] = child

# Final Result
best = max(population, key=fitness)

print("\nBest Solution:", best)
print("Fitness Value:", fitness(best))
```

Output:



iv. Recommended Approach

Recommended Algorithm: Genetic Algorithm

Justification

Scalability

- Efficiently handles hundreds of intersections.
- Can optimize many traffic signals simultaneously.

Adaptability

- Easily adapts to changing traffic conditions.
- Generates improved signal timings continuously.

Optimization Quality

- Finds near-optimal global solutions.
- Reduces congestion, waiting time, and fuel consumption more effectively than Hill Climbing.

Real-Time Performance

Although Genetic Algorithms require more computation, modern computers can execute them quickly enough for smart city traffic management.

Comparison

Criteria	**Hill Climbing**	**Simulated Annealing**	**Genetic Algorithm**
Scalability	Low	Medium	High
Adaptability	Medium	High	High
Solution Quality	Moderate	Good	Excellent
Real-Time Suitability	Good	Good	Very Good

Final Recommendation

The Genetic Algorithm (GA) is the most suitable approach for optimizing traffic signal timings in a smart city. It efficiently explores a large search space, avoids local optima through crossover and mutation, and continuously adapts to changing traffic conditions. While Hill Climbing is fast, it often gets trapped in local maxima. Simulated Annealing performs better by escaping local optima, but GA provides the best balance of scalability, adaptability, and solution quality for large-scale, real-time traffic optimization.

Conclusion

Traffic signal optimization is a complex optimization problem due to the large number of intersections and constantly changing traffic conditions. Traditional search methods are impractical because of the enormous search space. Hill Climbing is simple and fast but may get stuck in local maxima. Simulated Annealing improves exploration by accepting temporary worse solutions. Genetic Algorithms provide the most effective solution by exploring multiple candidate solutions simultaneously, making them the best choice for scalable and adaptive smart city traffic management.

Scenario 3: Autonomous Mars Rover (Online Search Agent)

Scenario Understanding

An autonomous rover is deployed on Mars to explore unknown terrain. Its objectives are to:

- Navigate safely to collect rock and soil samples.
- Avoid hazards such as craters, rocks, and steep slopes.
- Operate with limited sensing capability.
- Make decisions without a complete map.

Since the rover discovers new information while moving, it must continuously update its knowledge and make real-time decisions.

i. Why This Problem Requires an Online Search Agent

Online Search Agent

An online search agent makes decisions while exploring the environment. It does not require complete knowledge of the map before starting.

Why Online Search is Needed

- The rover has no complete map of Mars.
- New obstacles are detected only during exploration.
- The environment is uncertain and changes as the rover moves.
- It must react immediately to hazards while continuing its mission.

Example

If the rover detects a large crater while moving, it immediately changes its path instead of following the original route.

Advantages

- Works in unknown environments.
- Makes real-time decisions.
- Adapts to newly discovered obstacles.
- Improves mission safety.

ii. Challenges of Partial Observability and Dynamic Environments

1. Partial Observability

The rover cannot see the entire environment because its sensors have a limited range.

Challenges

- Unknown obstacles beyond sensor range.
- Limited information for decision-making.
- Risk of choosing unsafe paths.

Example

A large rock hidden behind a hill is detected only when the rover gets closer.

2. Dynamic Environment

The environment changes over time.

Challenges

- Dust storms reduce visibility.
- Loose soil may become unsafe.
- New hazards may appear.

Example

A safe route becomes blocked after a dust storm.

Impact

- The rover must continuously update its path.
- Decisions must be made quickly using the latest sensor information.

iii. How the Rover Builds and Updates Its Internal Model

Internal Model

The rover maintains an internal map of the environment using sensor data.

Steps

1. Move to a new location.
2. Collect data using cameras, LiDAR, and other sensors.
3. Identify safe and hazardous areas.
4. Update the internal map.
5. Select the next best path.
6. Repeat until the destination is reached.

Example

Initially:

Unknown Area

? ? ? ?

? R ? ?

? ? ? ?

After Exploration:

Safe Path

S S R

S X S

S S G

Where:

- **R** = Rover
- **S** = Safe path
- **X** = Obstacle
- **G** = Goal

The map becomes more accurate as the rover explores.

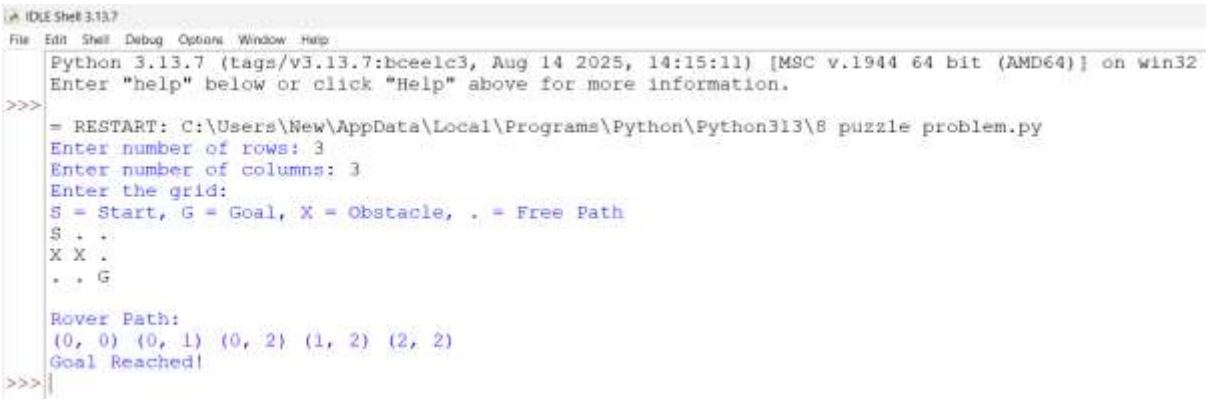
Python Code:

```
# Online Search Agent - Mars Rover

rows = int(input("Enter number of rows: "))
cols = int(input("Enter number of columns: "))

grid = []

print("Enter the grid:")
print("S = Start, G = Goal, X = Obstacle, . = Free Path")

for i in range(rows):
    row = input().split()
    grid.append(row)

# Find Start Position
```

```
for i in range(rows):
    for j in range(cols):
        if grid[i][j] == "S":
            x, y = i, j

visited = []
visited.append((x, y))

print("\nRover Path:")

while True:
    print((x, y), end=" ")

    if grid[x][y] == "G":
        print("\nGoal Reached!")
        break

    moved = False

    # Right
    if y + 1 < cols and grid[x][y + 1] != "X" and (x, y + 1) not in visited:
        y += 1
        moved = True

    # Down
    elif x + 1 < rows and grid[x + 1][y] != "X" and (x + 1, y) not in visited:
        x += 1
        moved = True

    # Left
    elif y - 1 >= 0 and grid[x][y - 1] != "X" and (x, y - 1) not in visited:
        y -= 1
        moved = True
```

```
    # Up
    elif x - 1 >= 0 and grid[x - 1][y] != "X" and (x - 1, y) not in visited:
        x -= 1
        moved = True

    if moved:
        visited.append((x, y))
    else:
        print("\nNo Path Found!")
        break
```

Output:



iv. Comparison of Online Search with Uninformed and Informed Search

Feature	**Online Search**	**Uninformed Search**	**Informed Search**
Environment Knowledge	Partial	Complete	Complete
Uses Heuristic	Yes (if available)	No	Yes
Suitable for Unknown Terrain	Yes	No	Limited
Real-Time Decision Making	Excellent	Poor	Moderate
Adaptability	High	Low	Medium
Safety	High	Moderate	High

Analysis

- **Uninformed Search** (BFS, DFS) assumes the environment is known and is inefficient for unknown terrain.
- **Informed Search** (Greedy Best-First Search, A*) uses heuristics but still generally assumes a known map.

- **Online Search** continuously learns and updates its decisions, making it ideal for Mars exploration.

v. Recommended Approach

Recommended Algorithm: Online Search Agent

Justification

Uncertainty

- Operates effectively without a complete map.
- Makes decisions using current sensor information.

Adaptability

- Updates routes whenever new obstacles are detected.
- Handles changing environmental conditions.

Safety

- Avoids hazardous areas immediately.
- Reduces the risk of collisions and mission failure.

Real-Time Performance

- Makes quick decisions without waiting to compute a complete path.
- Supports continuous exploration and sample collection.

Comparison

Criteria	**Offline Search**	**Online Search**
Complete Map Required	Yes	No
Real-Time Decisions	No	Yes
Unknown Environment	Poor	Excellent
Adaptability	Low	High
Safety	Moderate	High

Final Recommendation

The Online Search Agent is the most suitable approach for the Mars rover. It allows the rover to explore unknown terrain, update its internal model using sensor data, avoid hazards, and make safe real-time decisions. Unlike offline search methods, it does not require complete

knowledge of the environment and can adapt to uncertainty, making it ideal for autonomous space exploration.

Conclusion

The Mars rover operates in an uncertain and partially observable environment where a complete map is unavailable. Therefore, an Online Search Agent is the best solution because it continuously gathers information, updates its internal model, and adapts its path based on newly detected hazards. Compared to uninformed and informed search strategies, online search provides superior adaptability, safety, and real-time decision-making, ensuring successful and reliable autonomous exploration on Mars.

Scenario 4: University Exam Timetable Generation (Constraint Satisfaction Problem - CSP)

Scenario Understanding

A university wants to automatically generate an exam timetable using AI. The timetable must satisfy several constraints:

- No student should have overlapping exams.
- Limited number of exam halls.
- Some subjects must be scheduled before others.
- Faculty members must be available.
- Special accommodations must be be provided for certain students.

As the number of courses and students increases, the scheduling problem becomes more complex. Therefore, it is modeled as a Constraint Satisfaction Problem (CSP).

Problem Formulation as a Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem (CSP) consists of:

- Variables
- Domains
- Constraints

Variables

Variables represent the exams that need to be scheduled.

Example:

- E1 = Mathematics

- E2 = Physics
- E3 = Chemistry
- E4 = English

Domains

The domain is the set of possible values each variable can take.

Example:

- Time Slots = {Morning, Afternoon, Evening}
- Exam Halls = {Hall A, Hall B, Hall C}

Each exam must be assigned one valid time slot and hall.

Constraints

Hard Constraints (Must Be Satisfied)

- No student has two exams at the same time.
- Exam hall capacity must not be exceeded.
- One hall cannot host two exams simultaneously.
- Faculty must be available.
- Prerequisite subjects must be scheduled first.

Soft Constraints (Preferred)

- Provide sufficient gap between exams.
- Reduce consecutive exams for students.
- Minimize faculty workload.
- Schedule special accommodation students appropriately.

i. Variables, Domains, and Constraints

Component	Description
Variables	Exams (E1, E2, E3, E4...)
Domains	Available time slots and exam halls
Constraints	Student conflicts, hall limits, faculty availability, subject order, special accommodations

Explanation

The AI assigns a time slot and hall to each exam while ensuring that all constraints are satisfied. A valid timetable is one in which every exam has an assignment that does not violate any constraint.

ii. Backtracking Search

How Backtracking Works

Backtracking assigns values to variables one at a time. If a constraint is violated, it goes back (backtracks) and tries a different assignment.

Steps

1. Select an exam.
2. Assign a time slot and hall.
3. Check all constraints.
4. If constraints are satisfied, continue.
5. If a conflict occurs, backtrack and try another assignment.
6. Repeat until all exams are scheduled.

Example

Exam	**Assigned Slot**	**Result**
Math	Morning	✓
Physics	Morning	✗ Conflict
Physics	Afternoon	✓
Chemistry	Morning	✓

The algorithm changes the Physics exam to the afternoon after detecting a conflict.

Python Code:

```
# CSP - Exam Timetable using Backtracking

def is_safe(subject, slot, assignment):
    for sub in assignment:
        if assignment[sub] == slot:
            return False
```

```
    return True

def backtrack(subjects, slots, assignment, index):

    if index == len(subjects):
        return True

    subject = subjects[index]

    for slot in slots:

        if is_safe(subject, slot, assignment):

            assignment[subject] = slot

            if backtrack(subjects, slots, assignment, index + 1):
                return True

            del assignment[subject]

    return False


# Manual Input
n = int(input("Enter number of subjects: "))

subjects = []

print("Enter subject names:")
for i in range(n):
    subjects.append(input())

m = int(input("Enter number of available time slots: "))
```

```
slots = []

print("Enter time slots:")
for i in range(m):
    slots.append(input())

assignment = {}

if backtrack(subjects, slots, assignment, 0):

    print("\nExam Timetable")
    for subject in subjects:
        print(subject, "->", assignment[subject])

else:
    print("No valid timetable possible.")
```

Output:

iii. Constraint Propagation (Forward Checking)

What is Forward Checking?

Forward checking is a constraint propagation technique that removes invalid choices for future variables immediately after a value is assigned.

How It Improves Efficiency

- Detects conflicts early.
- Reduces unnecessary searching.
- Prevents impossible assignments.
- Speeds up timetable generation.

Example

If Mathematics is assigned to the Morning slot, and many students also take Physics, then Morning is removed from Physics' available time slots.

Remaining choices for Physics:

- Afternoon
- Evening

This avoids future conflicts and reduces backtracking.

Advantages

- Faster than simple backtracking.
- Reduces the search space.
- Finds valid timetables more efficiently.

iv. Real-World Challenges and Best Strategy

Real-World Challenges

- Thousands of students and courses.
- Limited exam halls.
- Faculty scheduling conflicts.
- Last-minute timetable changes.
- Special accommodations for some students.
- Maintaining fairness and minimizing student stress.

Best Strategy

The most effective approach is Backtracking Search with Forward Checking (Constraint Propagation).

Justification

Scalability

- Efficiently handles a large number of exams and constraints.

Adaptability

- Can adjust the timetable when new constraints are added.

Efficiency

- Forward checking reduces unnecessary searches and speeds up scheduling.

Accuracy

- Produces valid timetables while satisfying all hard constraints.

Comparison

Method	Advantages	Disadvantages
Simple Backtracking	Easy to implement	Slow for large problems
Backtracking + Forward Checking	Faster, fewer conflicts, efficient	Slightly more computation
CSP with Constraint Propagation	Highly scalable and accurate	More complex to implement

Final Recommendation

The Constraint Satisfaction Problem (CSP) approach using Backtracking Search with Forward Checking is the most suitable method for generating university exam timetables. It efficiently assigns exams to time slots and halls while satisfying all constraints. Forward checking reduces the search space by eliminating invalid assignments early, making the system faster, more scalable, and capable of handling large universities with many students and courses.

Conclusion

University exam scheduling is a complex CSP because multiple constraints must be satisfied simultaneously. Backtracking Search systematically assigns exams and revises decisions when conflicts occur, while Forward Checking improves efficiency by detecting conflicts early. Together, these techniques produce valid, scalable, and efficient exam timetables, making them the best choice for automated university scheduling.

Scenario 5: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)

Scenario Understanding

A gaming company is developing an AI opponent for a real-time strategy game. The AI must:

- Compete against human players.
- Make decisions within strict time limits.
- Evaluate many possible game states.
- Handle a very large decision tree efficiently.

The challenge is to choose the best move while maintaining fast response times. Minimax and Alpha-Beta Pruning are commonly used to solve such game-playing problems.

i. How the Minimax Algorithm Models This Problem

Minimax Algorithm

Minimax is an adversarial search algorithm used in two-player games. It assumes:

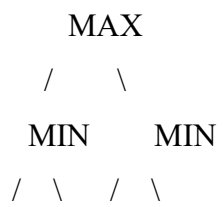
- The AI (MAX player) tries to maximize its score.
- The Human opponent (MIN player) tries to minimize the AI's score.

The algorithm explores possible future moves and selects the move that gives the AI the best guaranteed outcome.

Working Steps

1. Generate all possible moves.
2. Build a game tree.
3. Evaluate the outcomes of each move.
4. MAX chooses the highest value.
5. MIN chooses the lowest value.
6. Continue until the best move is found.

Example



3 5 2 9

- Left branch → MIN chooses **3**
- Right branch → MIN chooses **2**
- MAX chooses **3** (maximum of 3 and 2)

Advantages

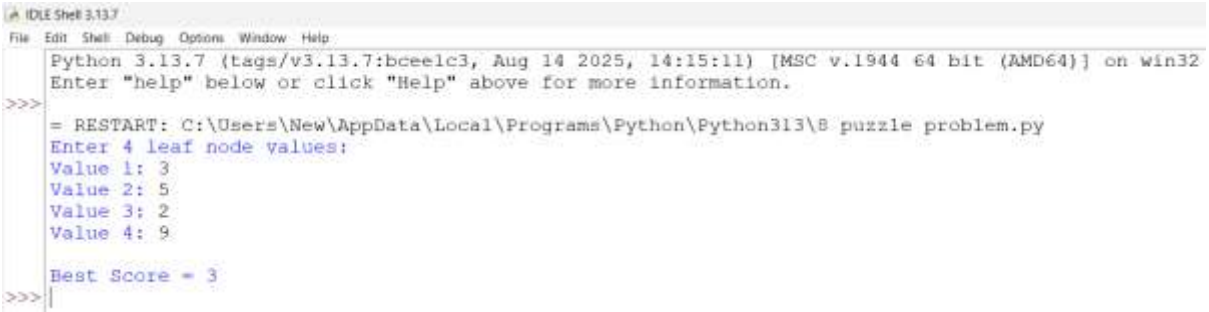
- Produces optimal decisions when the full game tree is explored.
- Considers the opponent's best possible moves.
- Suitable for turn-based strategic games.

Python Code:

```
import math

# Alpha-Beta Pruning Function
def alphabeta(depth, node, maximizing, values, alpha, beta):

    # Leaf node
    if depth == 2:
        return values[node]

    if maximizing:
        best = -math.inf

        for i in range(2):
            value = alphabeta(depth + 1, node * 2 + i, False,
                              values, alpha, beta)
            best = max(best, value)
            alpha = max(alpha, best)

            if beta <= alpha:
                break

        return best
```

```
    else:
        best = math.inf

        for i in range(2):
            value = alphabeta(depth + 1, node * 2 + i, True,
                              values, alpha, beta)
            best = min(best, value)
            beta = min(beta, best)

            if beta <= alpha:
                break

        return best


# Manual Input
print("Enter 4 leaf node values:")

values = []
for i in range(4):
    values.append(int(input(f"Value {i+1}: ")))

result = alphabeta(0, 0, True, values, -math.inf, math.inf)

print("\nBest Score =", result)
```

Output:



ii. Computational Challenges of Large Game Trees

In real-time strategy games:

- Each game state has many possible moves.
- Every move creates new game states.
- The number of possibilities grows exponentially (combinatorial explosion).

Challenges

- High computation time.
- Large memory usage.
- Difficult to search the entire game tree within time limits.
- Delayed responses reduce the quality of gameplay.

Example

If each state has 10 possible moves and the search depth is 5:

[
10^5 = 100,000 \text{ game states}
]

Evaluating all states is computationally expensive.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta Pruning

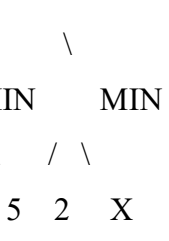
Alpha-Beta Pruning is an optimization of the Minimax algorithm. It eliminates branches that cannot influence the final decision, reducing unnecessary computations.

Key Values

- **Alpha (α):** Best value found so far for the MAX player.
- **Beta (β):** Best value found so far for the MIN player.

If **α ≥ β**, the remaining branch is pruned because it cannot improve the result.

Example



- Left MIN returns **3**.

- MAX stores **α = 3**.
- Right MIN finds **2**.
- Since **2 ≤ α**, the remaining branch (**X**) is pruned because MAX will never choose it.

Advantages

- Evaluates fewer game states.
- Produces the same optimal decision as Minimax.
- Reduces computation time.
- Improves response speed in real-time games.

iv. Evaluation Function

An evaluation function estimates the quality of a game state when it is not practical to search until the end of the game.

Factors Considered

- Number of units remaining.
- Health of units.
- Resources collected.
- Territory controlled.
- Strength of attack and defense.
- Distance to objectives.
- Strategic position on the map.

Example Evaluation Function

[
\text{Score} = (5 \times \text{Units}) + (4 \times \text{Resources}) + (3 \times \text{Territory}) - (2 \times \text{Enemy Strength})
]

A higher score indicates a better game state for the AI.

Justification

- Encourages stronger armies.
- Rewards resource collection.

- Promotes map control.
- Considers enemy threats for balanced decision-making.

v. Recommended Strategy

Recommended Approach: Minimax with Alpha-Beta Pruning

Justification

Optimality

- Minimax chooses the best move assuming the opponent also plays optimally.

Efficiency

- Alpha-Beta Pruning removes unnecessary branches without affecting the final decision.

Real-Time Performance

- Faster decision-making allows the AI to respond within strict time limits.

Scalability

- Can handle larger game trees more efficiently than standard Minimax.

Comparison

Criteria	**Minimax**	**Minimax + Alpha-Beta Pruning**
Decision Quality	Optimal	Optimal
Search Speed	Slow	Fast
Nodes Evaluated	High	Low
Memory Usage	High	Lower
Real-Time Suitability	Moderate	Excellent

Final Recommendation

The best approach is Minimax with Alpha-Beta Pruning. Minimax ensures the AI selects the best possible move while considering the opponent's strategy. Alpha-Beta Pruning significantly reduces the number of game states that must be evaluated, enabling faster decisions without sacrificing optimality. This combination provides the ideal balance between accuracy, efficiency, and real-time performance.

Conclusion

Real-time strategy games require AI to make intelligent decisions within very short time

limits. Minimax models the interaction between the AI and its opponent by selecting moves that maximize the AI's advantage while minimizing the opponent's opportunities. However, exploring every possible game state is computationally expensive. Alpha-Beta Pruning addresses this challenge by eliminating unnecessary branches, greatly improving efficiency while maintaining optimal decisions. Therefore, Minimax with Alpha-Beta Pruning is the most suitable approach for developing a fast, scalable, and intelligent game AI.